Trust-Explicit Resolution for Prediction Markets

Declared Logic, Verifiable Traces, and Dispute Economics

Andrei Savin

question.market

andrei@andreisavin.com

April 2026

Abstract

Prediction markets aggregate distributed knowledge into calibrated probability estimates. Their value depends on resolution: the process that determines which outcome actually occurred and who gets paid. In every major deployed prediction market today, this process is either centralized, opaque, or both. Participants bet not only on the outcome but on the platform's willingness and ability to determine the outcome correctly. The resolution rules may be vaguely defined, inconsistently applied, or changed after trading begins.

This paper studies a different approach: *trust-explicit* resolution. A trust-explicit protocol makes trust visible, bounded, and challengeable by committing each market to an immutable executable decision program, not only to a question string. The core mechanism is a *resolution blueprint*: a typed directed acyclic graph (DAG) of resolution steps, declared at market creation, stored immutably on-chain, and executed to produce an evidence trace that is reproducible in some cases and only trace-verifiable in others. Every market displays an automatically derived *trust class*, together with a more detailed trust profile, that tells participants what kind of judgment and fallback authority are involved before they trade.

The paper makes three contributions. First, it defines a formal trust stratification framework with five properties (declared, inspectable, reproducible, challengeable, decentralized) and maps existing prediction market resolution systems to this framework, while treating decentralization as a related execution property rather than as a substitute for declared logic. Second, it specifies a resolution-blueprint mechanism in which each market carries an executable resolution constitution: a typed DAG, stored immutably on-chain, compiled into a bounded runtime, and emitted as a challengeable execution trace with an evidence hash. Existing systems mostly specify who may answer and how disputes escalate; the blueprint mechanism instead specifies how the market itself is supposed to decide before any answer is proposed. Third, it presents a game-theoretic analysis of dispute economics: when it is rational to propose falsely, when it is rational to challenge, how bond sizing must scale with market value, why flat bonds are unsound for non-trivial markets, why verifier incentives and challenge probability matter more than adjudicator sophistication in thin markets, and why open proposing needs explicit success-conditioned compensation rather than unpaid labor. An agent-based simulation program supports the analysis across parameter sweeps covering bond sizing, challenge windows, participant attention, market scale, and proposer compensation.

The main findings are specific and operational. Proposer bonds must be proportional to market pool size, not flat constants. Verification coverage in thin markets is the primary failure mode, more dangerous than adjudicator error. Open proposing also needs explicit upside: returning the bond is not compensation for monitoring markets, running the blueprint, publishing evidence, and waiting through the challenge window. Challenge-linked verifier incentives and longer challenge windows materially improve integrity in the stylized model, while thin low-attention markets still require credible monitoring and prompt trace publication. Composition of independent evidence sources appears directionally helpful, though the current composition study is still proxy-level and hypothesis-generating rather than a full source-corruption model. The paper derives candidate deployment ranges for monitored markets and identifies the boundaries where those guarantees weaken.

1 Introduction **3**
1.1 What is new here . . . 4
1.2 Related work . . . 5

2 Why Resolution Is the Hard Problem **5**
2.1 What participants need to know . . . 5
2.2 Resolution in deployed systems . . . 5
2.3 Failure taxonomy . . . 6

3 Trust Stratification **7**
3.1 Declared . . . 7
3.2 Inspectable . . . 7
3.3 Reproducible, source-verifiable, and trace-verifiable . . . 7
3.4 Challengeable . . . 8
3.5 Decentralized . . . 8
3.6 Mapping deployed systems . . . 8

4 Resolution Blueprints **9**
4.1 Executable constitutions . . . 9
4.2 DAG model . . . 9
4.3 Step vocabulary . . . 10
4.4 Worked example: participant evidence + LLM . . . 10
4.5 Execution context and evidence traces . . . 11
4.6 On-chain storage . . . 12
4.7 Guarantee boundaries: contract, compiler, engine . . . 13
4.8 What a challenge actually attacks . . . 13

5 Trust Classes **14**
5.1 Classification function . . . 14
5.2 What the classes mean . . . 15
5.3 LLM judges: bias, composition, and credibility . . . 15

6 Dispute Economics **16**
6.1 The resolution game . . . 16
6.2 When is it rational to propose falsely? . . . 16
6.3 When is it rational to challenge? . . . 17
6.4 The lazy verifier problem . . . 17
6.5 Verifier incentives . . . 18
6.6 Proposer compensation . . . 19
6.7 Bond sizing . . . 19

6.8 Challenge window . . . 21
6.9 Adjudicator incentives . . . 21
6.10 Escalation architecture . . . 22

7 Simulation Program **22**
7.1 Agent types . . . 22
7.2 Parameters . . . 23
7.3 Metrics . . . 23
7.4 Scenario families . . . 24

8 Results **24**
8.1 Flat bonds fail on large markets . . . 24
8.2 Pool-proportional bonds improve scale behavior, but do not secure thin markets by themselves 25
8.3 Verification coverage is the primary failure mode . . . 25
8.4 Stylized verifier incentives improve thin-market coverage . . . 26
8.5 Challenge window has diminishing returns past 24 hours . . . 26
8.6 Adjudicator accuracy matters less than challenge probability . . . 27
8.7 Escalation structure comparison . . . 27
8.8 Position bounds help only when exposure is genuinely diffuse, and large-pool liveness remains a proposer-capital problem . . . 28
8.9 Compensation converts capital eligibility into actual proposer liveness . . . 28
8.10 Preliminary composition proxy . . . 29

9 What the Results Mean **29**

10 Protocol Implications **30**

11 Deployment Boundaries **31**

12 Conclusion **32**

1 Introduction

Prediction markets work because participants risk capital on outcomes. A share in "Team A wins" that pays $1 if correct and $0 otherwise carries economic consequence for the buyer. That consequence is what separates a prediction market from a poll. It is also why resolution, the process that determines which shares pay out, is the most important part of the system. Everything else (pricing, liquidity, order matching) is infrastructure. Resolution is where participants' money actually changes hands.

This would be unremarkable if resolution were a solved problem. It is not. In the largest deployed prediction market as of early 2026, Polymarket, resolution works as follows: the platform team defines the question, an external oracle system (UMA's Optimistic Oracle) accepts bonded proposals, and if nobody disputes within a window, the proposal finalizes [3]. The mechanism has useful properties: it is bonded, it is challengeable, and it has an escalation path. But the question itself, the thing participants are betting on, is drafted by a centralized team, and participants have no protocol-level guarantee about how future questions will be worded, interpreted, or scoped. If the platform changes its editorial policy, the resolution norms change with it. There is no immutable specification to audit.

Other systems make different tradeoffs. Reality.eth provides a bonded-answer primitive where anyone can propose an answer, subsequent proposals require larger bonds, and disputes escalate to Kleros as an external arbitrator [4]. The mechanism is more decentralized than a platform-controlled oracle, but the resolution logic is external to the market: the market references an oracle contract but does not itself declare how truth will be determined. Centralized platforms resolve by editorial discretion, which is the fastest and cheapest approach and also the one that provides the least structural accountability.

The thesis of this paper is that a stronger property is achievable and worth pursuing. We call it *trust-explicit* resolution. Every prediction market resolution system depends on some external input about what actually happened in the world. The design goal is therefore not to remove trust, but to bind it to declared procedures that participants can inspect before trading and challenge after execution. Markets should commit to an executable decision program, not just to a question string or an oracle brand. A system that declares that program at market creation, stores it immutably, executes it to produce a verifiable trace, and backs the result with bonded dispute economics provides a stronger guarantee along an explicitly defined trust stratification than the alternatives.

The contribution has three parts. Section 3 defines a trust stratification framework with five properties and maps existing systems to it. Sections 4 and 5 describe the resolution blueprint mechanism: a typed DAG with a minimal step vocabulary, automatic trust classification, and on-chain storage. Section 6 presents the paper's main analytical contribution: a game-theoretic model of dispute economics that derives bond sizing requirements, identifies the lazy verifier problem, and analyzes challenge-linked verifier incentives. Section 7 describes an agent-based simulation program that tests the analytical results across market conditions. Section 8 presents findings.

This contribution is orthogonal to the trading mechanism. LMSR-style automated market makers [1], order-book exchange designs, and conditional-token architectures can each support trading, but none of them by themselves answer the resolution question. Conditional tokens, for example, already formalize a condition as "a question to be answered in the future by a specific oracle" [9]; the blueprint proposal extends that idea from a single oracle hook to an executable and challengeable resolution program.

1.1 What is new here

Disputes, optimistic oracles, and arbitration backstops are already part of the deployed ecosystem. Systems such as Polymarket/UMA and Reality.eth/Kleros already show that bonded proposals, challenge windows, and escalation can work in production. The novelty claim here is narrower and more specific: it concerns *what is being declared and analyzed*.

Three pieces matter:

1. **Blueprint formalism.** Existing systems usually specify who may answer and how disputes escalate. The blueprint mechanism specifies how the market itself is supposed to decide before any answer is proposed.
2. **Trust semantics.** The trust class/profile vocabulary turns the resolution trust surface into protocol-visible state rather than leaving it embedded in natural-language norms.
3. **Resolution economics.** The paper treats resolution as its own mechanism-design problem, with the lazy-verifier problem and bond scaling as primary constraints rather than as operational details downstream of trading.

The paper's claim is that the combination of declared logic, verifiable traces, and calibrated dispute economics produces a resolution system whose trust properties are formally characterizable, empirically testable, and stronger along the paper's defined trust stratification than those of current deployed alternatives.

1.2 Related work

This paper sits at the intersection of prediction-market resolution, oracle design, and decentralized arbitration. Deployed systems such as UMA/Polymarket and Reality.eth/Seer show that bonded proposals, challenge windows, and escalation can work in practice [3, 4, 6, 7]. Kleros and related arbitration systems contribute the external-judgment layer used when a bonded-answer game cannot settle a dispute on its own [5]. Augur's dispute-fork design and Schelling-point approaches such as Schelling-Coin push further toward subjective-truth consensus, but still focus primarily on who resolves and how consensus forms rather than on exposing an executable decision workflow at market creation [8, 13]. Conditional tokens are adjacent in a different direction: they formalize oracle-addressed payout conditions, but stop short of storing a richer decision program inside the market itself [9]. The contribution here is complementary to that literature: a resolution-language design, a trust taxonomy, and a dispute-economics analysis for declared decision workflows.

2 Why Resolution Is the Hard Problem

2.1 What participants need to know

Before placing a trade, a rational participant in a prediction market needs four pieces of information about the resolution process:

1. **What** will determine the outcome. Is it an API data feed, a human judgment, an AI evaluation, or some combination?
2. **Who** will execute the determination. Is it the platform, a designated oracle, any willing party, or an automated process?
3. **What happens if the determination is wrong.** Is there a dispute mechanism? What does it cost to trigger? Who adjudicates?
4. **What recourse exists.** If the dispute mechanism itself fails, what is the fallback?

Most deployed prediction markets answer question (1) vaguely and questions (2) through (4) not at all. A participant must infer the resolution norms from the platform's track record, terms of service, and community reputation. The result is structural opacity: the information exists but is not available in a form that can be inspected, compared, or enforced at the protocol level.

2.2 Resolution in deployed systems

Three resolution architectures dominate current practice.

Platform-controlled resolution. A centralized team drafts the question, interprets the outcome, and resolves the market. This is fast, cheap, and administratively simple. The failure mode is discretionary risk: the same team that benefits from market activity controls the resolution. Even without bad intent, interpretation of ambiguous outcomes introduces inconsistency that participants cannot predict or challenge structurally.

Oracle-delegated resolution. The market delegates resolution to an external oracle contract. UMA's Optimistic Oracle, used by Polymarket, is the most prominent example [3]. A proposer bonds a claim about the outcome; if nobody disputes within a window, the claim finalizes. Disputes escalate to UMA token-holder voting. The mechanism provides economic accountability: false proposals cost money. But the market's question specification is still controlled by the platform, and the oracle's interpretation of that specification is a separate trust assumption layered on top.

Bonded-answer escalation. Reality.eth allows anyone to propose an answer to a question, with each subsequent proposal requiring a higher bond [4]. If no higher-bonded counter-proposal arrives within the timeout, the last answer wins. Disputes escalate to Kleros, a decentralized arbitration protocol [5]. This is the most decentralized resolution approach in production. The resolution logic, however, is external to the market. A market on Seer references a Reality.eth question, but the market itself does

Table 1. What different systems commit to at market creation time. The central claim of this paper is that resolution should commit a decision program, not only a question string or oracle reference.

System	Immutable object at creation	What remains outside the commitment
Platform-controlled	Question text and platform policy references	Editorial interpretation and future resolution norms
UMA / Polymarket style	Question string plus external oracle path	The executable decision logic that produces the answer
Reality.eth / Seer style	Question string plus bonded-answer / arbitration reference	The decision workflow beyond the oracle game itself
Blueprint market	Question specification plus main/dispute blueprint bytes and fallback path	Only the declared external reality inputs, not the resolution logic itself

not declare how the answer will be determined. The resolution specification is a natural-language question string, not a structured program.

The distinction is easiest to see at market creation time: what exactly becomes immutable before trading begins?

2.3 Failure taxonomy

Resolution failures are not all the same. Six distinct failure modes affect prediction market resolution:

Failure Mode	Description	Primary Exposure
Ambiguous specification	The question admits multiple valid interpretations	All systems using natural-language questions
Interpretive discretion	The resolver chooses one interpretation without structural accountability	Platform-controlled resolution
Data source failure	The oracle or API that provides ground truth is down, wrong, or manipulated	Oracle-delegated and bonded-answer systems
Executor malfeasance	The party executing the resolution submits a deliberately false outcome	Any system with a single resolution executor
Liveness failure	Nobody resolves the market (the resolver is offline, incentives are insufficient)	Systems with designated resolvers
Escalation capture	The dispute adjudicator is compromised or colluding with one party	Bonded-answer systems with token-holder voting

No single system is immune to all six. The relevant question is not "which system eliminates all failure modes" but "which system makes its failure exposures visible and provides structural countermeasures for the most dangerous ones."

One point deserves emphasis because it sits upstream of every dispute game: question quality. Forecasting platforms with strong editorial discipline explicitly require authoritative sources, explicit edge-case handling, and fallback criteria in their resolution text [10]. A blueprint cannot rescue a malformed or semantically incoherent market specification. The relevant design object is therefore the pair *(question specification, blueprint)*, not the blueprint in isolation. In practice that means the question side should also become more structured over time: typed outcome domains, explicit temporal scope, declared tie-breaking rules, named fallback-to-cancel conditions, and stated handling for foreseeable edge cases. Blueprint authoring quality belongs in the same upstream threat model.

Stronger declaration makes bad logic legible, but it also makes that logic immutable once trading begins. Compiler and runtime validation can reject malformed programs; they cannot guarantee that a declared workflow is economically sensible, semantically fair, or robust to adversarial wording.

3 Trust Stratification

We define five properties of a resolution system. The first four represent a structural ladder of guarantees that can be verified independently; the fifth, decentralization, is a related execution property rather than a strict continuation of that ladder.

3.1 Declared

A resolution system is *declared* if the resolution method is specified at market creation and cannot be changed after the first trade is placed. Let R_m denote the resolution specification for market m, and let t_0 denote the time at which the first trade occurs. Declaredness requires:

$$\forall\, t > t_0 : R_m(t) = R_m(t_0).$$

This is a simple immutability condition. It prevents the resolution rules from being rewritten after participants have committed capital. A platform that reserves the right to change its resolution criteria after trading begins does not satisfy this property, regardless of whether it actually exercises that right.

3.2 Inspectable

A resolution system is *inspectable* if any participant can read R_m and understand exactly what the resolution process will do. For a natural-language specification ("the market resolves based on the official result"), inspectability depends on the precision of the language. For a formal specification (a program, a DAG, a decision tree), inspectability depends on whether the specification is public and whether its semantics are defined.

Inspectability is not a binary property. A resolution blueprint stored as a typed DAG with known step semantics is more inspectable than a prose description referencing an external oracle. The distinction matters because participants are pricing resolution risk into their trades. More inspectable resolution logic reduces the uncertainty premium.

In practice, inspectability is mediated through tooling. Most participants will not read a raw DAG before trading. They consume trust classes, trust profiles, validation warnings, summaries, and creator history, while retaining the ability to inspect the canonical blueprint if they want to go deeper. Transparent availability of the blueprint does not guarantee that every participant will inspect it in full, but it enables three defenses absent in opaque systems: tooling can summarize and classify the trust surface, frontends can rank and filter markets by blueprint quality, and creators accumulate a durable reputation tied to declared resolution design. These are practical defenses, not complete ones. They complement compiler/runtime validation and protocol-level constraints; they do not replace them.

3.3 Reproducible, source-verifiable, and trace-verifiable

A resolution system is *reproducible* if, given the same inputs, an independent party can rerun the resolution procedure and obtain the same result. In practice, replayability comes in tiers rather than a binary:

- A *deterministic* step (reading from a fixed log, evaluating a logical condition, computing a hash) is fully reproducible: given the same inputs, any executor will produce the same output.
- A *source-based* step with replayable evidence (a signed message, archived API response, fixed log, or timestamped external record) is weaker than full determinism but stronger than bare trace-verifiability. An independent party may not recreate the world event itself, but it can verify the source object and the blueprint's declared extraction logic.

- A *non-deterministic or non-replayable* step (fetching a live unsigned API, querying a closed-model LLM, invoking a human judge) is *trace-verifiable*: the inputs and outputs are recorded in the execution trace, so an independent party can verify *what the executor reported seeing and deciding*, even if it cannot independently recreate the same API response or LLM output.

Trace-verifiability is weaker than full reproducibility, but it is much stronger than no verification at all. An execution trace that records every node's inputs, outputs, and intermediate state constitutes an audit log. A false trace is a falsifiable claim: the proposer asserts that specific inputs produced specific outputs through a specific computation. If any step in the trace is inconsistent with the declared blueprint, the trace is invalid and the proposal is challengeable. Replayable source-based blueprints can satisfy full reproducibility or strong source-verifiability. Blueprints that depend on live unsigned APIs, closed-model LLMs, or human judges generally satisfy only trace-verifiability, and the paper treats that distinction as substantive rather than semantic.

3.4 Challengeable

A resolution system is *challengeable* if any participant can dispute the proposed outcome with economic commitment and a defined adjudication path. Three components are required:

1. A *challenge bond*: the challenger posts capital that is forfeited if the challenge fails.
2. A *challenge window*: a time period during which challenges can be submitted.
3. An *adjudication path*: a predeclared process for resolving the dispute.

Challengeability without inspectability is weak: if participants cannot understand the resolution logic, they cannot evaluate whether a proposal is correct. Challengeability without reproducibility is weaker still: without a verifiable trace, a challenger must argue from outside evidence rather than pointing to a specific step in the execution that produced the wrong output.

3.5 Decentralized

A resolution system is *decentralized* if no single party can unilaterally determine the outcome. This is a spectrum. At one end: a single platform operator clicks a button. At the other: a fully permissionless bonded-answer game with token-holder arbitration. Most systems fall in between.

Decentralization of resolution execution is distinct from decentralization of resolution logic. The first four properties above form the main epistemic and procedural ladder: what is fixed, what is legible, what is replayable, and what can be challenged. Decentralization is a related execution property rather than a simple capstone on that ladder. A system in which a centralized operator executes a declared, inspectable, challengeable resolution blueprint with verifiable traces is more trust-explicit than a system in which a decentralized oracle executes an unspecified resolution process. But this claim is conditional on verification coverage: centralized execution is defensible only when the logic, inputs, outputs, and challenge mechanism make the run verifiable *and* the market expects enough monitoring that challenges are not purely theoretical.

3.6 Mapping deployed systems

Table 3 maps existing resolution approaches to the five properties.

The partial marks warrant explanation. UMA and Reality.eth are "partially declared" because the question text is fixed at creation but the resolution interpretation is not formally specified. They are "partially inspectable" because participants can read the question but cannot inspect the resolution program. A full Declared score would require immutable formal semantics for the resolution workflow, not only immutable question text; a full Inspectable score would require participants to inspect that workflow rather than infer it from prose and oracle reputation. Neither provides reproducibility because neither produces a structured execution trace. Replayable source-based blueprints score full

Table 3. Trust stratification of prediction market resolution systems. ✓ indicates the property is satisfied; ∼ indicates partial satisfaction; – indicates the property is not satisfied. Here ∼ means that a market fixes a public question or oracle reference but not a fully specified executable resolution procedure, or that it provides weaker source/trace verifiability rather than full reproducibility. The first four columns form the main declared-to-challengeable ladder; decentralization is a related execution axis.

System	Declared	Inspect.	Replayability	Challenge.	Decentr.
Platform-controlled	–	–	–	–	–
UMA Optimistic Oracle	∼	∼	–	✓	∼
Reality.eth + Kleros	∼	∼	–	✓	✓
Blueprint (replayable source-based)	✓	✓	✓	✓	∼
Blueprint (live-source / agent / human)	✓	✓	∼	✓	∼

marks on the first four properties and partial on decentralization: the logic is fully declared and verifiable, but execution in the initial deployment is centralized. Blueprints that depend on live APIs, LLM judges, or human judgment retain the same declared, inspectable, and challengeable structure, but their non-deterministic or non-replayable steps are better described as source-verifiable or trace-verifiable than fully reproducible. The paper's argument is that these structural properties are more important than premature decentralization of execution, because they make even centralized execution auditable and challengeable.

4 Resolution Blueprints

A resolution blueprint is a directed acyclic graph of typed steps that defines exactly how a market's question will be answered. This section defines the mechanism formally.

4.1 Executable constitutions

The central claim is that each market carries an *executable constitution*: a finite program that fixes, before trading begins, what evidence will be gathered, which branch conditions determine whether the market resolves, defers, or cancels, and which fallback path is taken if the main path does not produce a valid answer.

In the current implementation a market stores two such programs on-chain: a *main blueprint* and a *dispute blueprint*. Both must be present before bootstrap. The resolution runner executes the main blueprint first; if that path fails, returns only a defer/cancel action, or is later challenged, the dispute blueprint is the predeclared escalation path. This is materially stronger than saying only that "resolution uses oracle X" or "the platform will decide." The market commits to the decision flow itself.

4.2 DAG model

A resolution blueprint is a tuple $G = (V, E, \tau, \gamma, \beta)$ where:

- $V = \{v_1, \ldots, v_n\}$ is a finite set of nodes (resolution steps).
- $E \subseteq V \times V$ is a set of directed edges.
- $\tau : V \to T$ maps each node to a step type from the type vocabulary T.
- $\gamma : E \to C$ maps each edge to a condition expression, where C is a set of boolean-valued expressions over the execution context.
- $\beta = (t_{\max}, k_{\max})$ is a budget constraint: maximum execution time $t_{\max}$ and maximum LLM token consumption $k_{\max}$.

The graph must be acyclic in its forward edges. (Back-edges with bounded traversal counts can support retry patterns without introducing unbounded execution.) Execution proceeds from root nodes (those with no incoming forward edges) toward terminal nodes. At each node, the executor runs the step, writes outputs to a shared context, and then evaluates outgoing edge conditions to determine which successors activate. If multiple root nodes exist, they may execute in parallel.

4.3 Step vocabulary

The type vocabulary T is intentionally small. Eight step types compose into expressive resolution workflows:

Step Type	Description	Trust Model	Deterministic
`api_fetch`	Fetch a URL, extract a value, map to an outcome	Source trust	No
`market_evidence`	Load signed evidence submissions from market participants	Source-based	Yes
`llm_judge`	Present evidence to an LLM and obtain a structured verdict	Probabilistic	No
`human_judge`	A human or panel determines the outcome	Subjective	No
`wait`	Pause execution or defer against a time anchor	None	Yes
`defer_resolution`	End the current run without proposing	None	Yes
`submit_result`	Submit the winning outcome	Terminal	Yes
`cancel_market`	Cancel and trigger refunds	Terminal	Yes

The vocabulary is minimal by design. A small vocabulary keeps blueprints auditable: participants can understand the complete resolution process because there are only eight possible step types. In the current design, additional behaviors should reduce either to composition of these steps or to cross-run control flow; if a proposed extension cannot be expressed that way, it should justify itself as a genuinely new trust surface rather than as convenience syntax.

Steps compose via directed edges with conditional routing. A typical blueprint might specify: wait for a 12-hour evidence window; if the window is still open, defer and retry later; once it closes, load participant evidence, present it to an LLM judge, and submit or cancel depending on whether the result is conclusive.

These patterns are representative rather than exhaustive, but they cover the main resolution shapes used in the current preset family. The design choice that keeps the vocabulary small is to treat repeated waiting and backoff as cross-run control flow rather than as internal graph cycles. That preserves the auditable DAG structure while still expressing the practical resolution loops a production market needs.

4.4 Worked example: participant evidence + LLM

One V1 preset makes the mechanism concrete. The `participant_evidence_llm` blueprint begins with a `wait` node anchored to `resolution_pending_since`. If the evidence window is still open, execution terminates with `defer_resolution` and the engine retries later. Once the window closes, the blueprint loads the signed participant-evidence bundle, passes that bundle to an `llm_judge` node, and either submits a conclusive outcome or cancels the market if the evidence remains contradictory or insufficient.

In compact routing form:

Table 5. Representative resolution patterns expressible with the V1 step vocabulary. Retries remain acyclic because a run terminates with `defer_resolution` and is restarted later against updated time anchors or context.

Pattern	Blueprint sketch	How the vocabulary expresses it
Timed evidence collection	`wait` → `market_evidence` → `llm_judge` → `submit/cancel`	Conditional edges capture whether the evidence window is still open and whether the judge reaches a conclusive outcome.
First usable source among several APIs	`api_fetch(A)` → `submit` / `api_fetch(B)` → `submit` / `api_fetch(C)` → `cancel`	Fallback order is encoded directly in edge conditions, so the graph can express "try A, else B, else C" without additional step types.
Conditional retry with backoff	`wait` → `api_fetch` → `defer_resolution`	Backoff is externalized across runs: the graph stays acyclic, while the runner updates the next eligible execution time or anchor state before re-entry.
Multi-source cross-check	`api_fetch` + `market_evidence` + `llm_judge` → `submit/cancel`	Agreement or consistency conditions live on the outgoing edges, allowing the blueprint to require corroboration before submitting a result.
Dispute escalation with independent logic	main blueprint → challenge → dispute blueprint → final fallback	Expressiveness comes from composing two declared DAGs: a fast main path and a separately declared dispute path with different evidence sources or adjudication steps.

1. if `wait.status = waiting`, the run terminates with `defer_resolution`;
2. if `wait.status = success`, the run advances to `market_evidence`;
3. if the evidence bundle loads successfully, the run advances to `llm_judge`; otherwise it cancels;
4. if `judge.outcome` is conclusive, the run reaches `submit_result`; otherwise it reaches `cancel_market`.

This is exactly the kind of structure that is hard to express with a single oracle reference. The participant can inspect not only *who* may decide, but the entire decision flow: evidence window, evidence loader, judging model, cancellation rule, and retry behavior. The market's headline trust class is `agent_assisted`; the fuller trust profile also includes the declared evidence source, defer semantics, and dispute fallback.

4.5 Execution context and evidence traces

Execution produces a flat key-value context $\Gamma : \text{String} \to \text{String}$, where each node v_i writes its outputs with prefix $v_i.key$. Edge conditions are evaluated against Γ using a standard expression language. The complete context at termination is accompanied by node statuses, edge traversals, and per-node input/output snapshots. Together these objects constitute the *evidence trace*: a record of every node's inputs, outputs, and the path taken through the graph.

The terminal `submit_result` step computes an evidence hash

$$H = \text{SHA-256}(\text{serialize}(\Gamma))$$

over the context snapshot and submits it on-chain alongside the proposed outcome. This pins the trace contents: a verifier can request the full trace, recompute the hash, and confirm that it matches

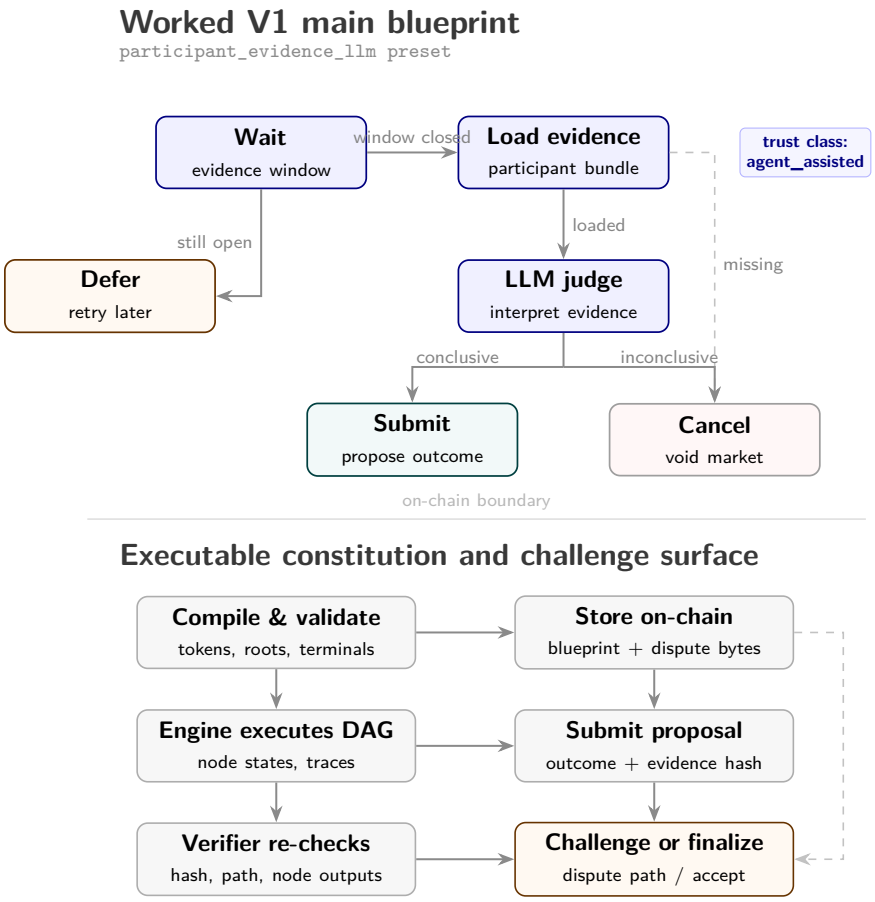

Figure 1. A worked V1 blueprint and the associated challenge surface. The main path is an actual preset structure used by the blueprint tooling: wait for the evidence window, load participant evidence, judge, then submit, cancel, or defer. The right-hand side shows what makes the blueprint an executable constitution rather than mere documentation: compilation and validation, immutable on-chain commitment, runtime traces, evidence hashing, and a defined challenge path.

the on-chain commitment. Fabricating a trace that produces the correct hash but contains different content requires either a hash collision or an execution trace that the verifier cannot independently challenge.

The runtime records a richer object than Γ alone. For each node execution it stores a node status, an input snapshot, outputs, timestamps, and resource usage. During execution the runner can emit defensive run snapshots to an indexer, so a verifier need not trust an end-of-run summary assembled after the fact. This is the operational core of trace-verifiability: the market publishes not only an answer, but an auditable path to that answer.

Hashing alone is not enough. A proposal should be treated as valid only if the full trace and referenced evidence objects are made available during the challenge window in a form challengers can retrieve. In operational terms, this is a "no trace, no finalization" rule: if the proposer cannot publish the complete trace, the proposal is itself challengeable or automatically invalid. Hash commitment protects integrity; trace availability protects challengeability.

4.6 On-chain storage

The blueprint is stored on-chain in the market contract's box storage, not on IPFS or any external system. Users are betting capital on the resolution logic. That logic must be as available and verifiable

Table 6. Where blueprint guarantees come from in the current implementation.

Layer	What it guarantees today	Important caveat
Contract	Stores main and dispute blueprint bytes immutably during `CREATED`; requires both boxes to be present before bootstrap; binds proposal, challenge-window, and resolution state on-chain.	The contract stores opaque bytes. It does not yet perform full JSON/schema/DAG validation on-chain.
Compiler / editor	Resolves market-specific tokens, strips editor state, enforces the 8 KB serialized-size cap, restricts node types, requires roots and terminal paths, rejects dangling edges and self-loops, and requires bounded back-edges.	These are off-chain checks prior to submission. They improve safety and legibility, but they are not yet consensus-enforced.
Engine / runner	Revalidates the blueprint, requires a registered executor for every node type, enforces time/token budgets, records per-node traces and edge traversals, computes the evidence hash, and supports predeclared main/dispute execution paths.	V1 execution is still operationally centralized. Integrity therefore comes from immutable commitment plus trace-and-challenge, not from trustless execution.

as the contract itself. The blueprint hash (SHA-256 of the serialized JSON) is stored in global state as an integrity check.

This satisfies the Declared and Inspectable properties by construction. The blueprint is immutable after market creation (the contract does not expose a method to modify it), and anyone can read it from box storage and verify the hash.

4.7 Guarantee boundaries: contract, compiler, engine

The current system divides responsibility across three layers, with each layer making a different kind of guarantee.

This decomposition is worth stating explicitly because it clarifies what the differentiator really is. The differentiator is not "a DAG editor" in the abstract. It is the combination of an immutable on-chain commitment, a typed and validated authoring layer, and a runtime that produces falsifiable execution traces under a declared fallback path.

That distinction matters for the paper's guarantees. "Declared" in V1 means immutable on-chain bytes. "Executable" and "safe by construction" are stronger statements that currently depend on canonical tooling and runtime validation. The present design therefore combines consensus-enforced byte commitment with off-chain validation and runtime checks, rather than treating those as one undifferentiated guarantee.

4.8 What a challenge actually attacks

A challenge is not a vague appeal to platform discretion. It is a claim that some executable object is wrong. In practice there are five main challenge targets:

1. **Commitment mismatch.** The trace or proposal refers to blueprint bytes whose hash does not match the on-chain commitment.
2. **Trace mismatch.** The proposed outcome is accompanied by an evidence hash that does not match the published execution trace.
3. **Deterministic-step error.** A deterministic node (`wait`, `market_evidence`, routing logic, or

Table 7. Validity layers for a blueprint commitment. This separates immutable bytes from stronger claims such as well-formedness or consensus-enforced validity.

Claim about the market	What it actually means	Enforced today by
Immutable bytes	The exact submitted blueprint bytes cannot change after market creation.	Consensus / contract
Well-formed blueprint	The bytes deserialize into a known schema version with valid node types and field structure.	Compiler / editor and engine revalidation
Canonical commitment	The hash refers to a canonical serialized object with domain separation and version tags rather than to ad hoc JSON formatting.	Tooling convention today; should become part of the compiler/runtime contract
Runtime-valid executable blueprint	The graph has roots, terminals, bounded retry structure, registered executors, and a budget-valid execution path.	Engine / runner
Consensus-enforced validity	The chain itself rejects malformed blueprints or invalid finalization attempts.	Only partially: bytes and state transitions are consensus-enforced; full blueprint validity is not yet

outcome mapping) produced an output inconsistent with the declared inputs and semantics.

4. **Path-selection error.** The trace took a branch or terminal action not justified by the recorded node outputs.
5. **Non-deterministic-step dispute.** A non-deterministic node (`api_fetch`, `llm_judge`, or `human_judge`) produced an output that a verifier contests with independent evidence or with the dispute blueprint.

This is why the blueprint engine and the bond mechanism are complements rather than substitutes. The engine turns "the resolution was wrong" into a narrower falsifiable statement about bytes, traces, path selection, or node outputs. The bond-and-challenge game then prices that falsifiable claim. This narrowing of the dispute surface is the main practical payoff of committing a market to an executable decision program rather than only to a question string.

5 Trust Classes

Not all blueprints carry the same trust assumptions. A blueprint that resolves from declared source data requires trust in the source and, unless the source object is signed or archived, in the executor's honesty about what the source returned. A blueprint that includes an LLM judge requires trust in the model's judgment and the integrity of the model-serving path. A blueprint that includes a human judge requires trust in the human's honesty and competence. These are materially different risk profiles, and participants should know which one applies before they trade.

5.1 Classification function

Define the trust class function $\kappa : \mathcal{G} \to \{\texttt{source_based}, \texttt{agent_assisted}, \texttt{human_judged}\}$ as follows. Let $T(G) = \{\tau(v) : v \in V\}$ be the set of step types present in blueprint G.

$$\kappa(G) = \begin{cases} \texttt{human_judged} & \text{if } \texttt{human_judge} \in T(G) \\ \texttt{agent_assisted} & \text{if } \texttt{llm_judge} \in T(G) \\ \texttt{source_based} & \text{otherwise} \end{cases}$$

The classification is monotone: adding a node of a higher trust class to a blueprint never lowers the classification. Formally, if $G' = (V \cup \{v'\}, E', \tau', \gamma', \beta)$ extends G by adding a node v', then $\kappa(G') \geq \kappa(G)$ in the ordering $\texttt{source_based} < \texttt{agent_assisted} < \texttt{human_judged}$.

The scalar trust class is intentionally coarse. It is useful for scanning markets quickly, but by itself it does not exhaust the trust surface. In particular, it does not encode the adjudication path, the final fallback authority, or whether the blueprint is fully reproducible or only trace-verifiable. A production system should therefore publish a *trust profile* alongside $\kappa(G)$:

$$\chi(G) = \big(\kappa(G),\ a(G),\ f(G),\ r(G)\big)$$

where $a(G)$ identifies the adjudication path, $f(G)$ identifies the final fallback, and $r(G) \in \{\texttt{reproducible}, \texttt{trace_verifiable}\}$ records whether the blueprint can be independently rerun or only independently audited. The class remains the headline label for listings; the profile is the more operational object and carries the finer-grained trust assumptions.

5.2 What the classes mean

Source-based. The blueprint contains only deterministic, source, and evidence-loading nodes. Resolution depends on declared inputs and extraction logic rather than on an explicit judgment node. Some source-based blueprints are fully reproducible because the source objects are fixed, signed, or archived; others are only source-verifiable or trace-verifiable because they depend on live unsigned APIs or executor-reported fetches.

Agent-assisted. The blueprint includes an LLM judge. An AI model participates in the resolution process. The model identifier, prompt, and decision criteria are declared in the blueprint, so the trust assumption is at least inspectable. But inspectable does not mean fully auditable: provider-side changes, hidden system prompts, refusals, prompt injection, and model deprecation all remain relevant risks unless the serving path is tightly pinned and signed.

Human-judged. The blueprint includes a human judge. A human makes the final determination. This is the highest trust class because human judgment is the least reproducible and the most subject to undisclosed bias. It is also sometimes the only viable resolution method for genuinely subjective questions.

Trust classes are displayed prominently in the market interface. A participant can decide whether they are comfortable with the resolution process before they trade. This is the core of trust explicitness: not the absence of trust, but the visibility of what trust is required. The class is the headline. The executable constitution and the trace make the headline inspectable.

5.3 LLM judges: bias, composition, and credibility

An LLM judge is probabilistic. Given the same evidence, two runs of the same model may produce different outputs. The prompt, model version, and temperature are declared in the blueprint, which makes the judgment *inspectable* in a way that human judgment is not. But inspectability does not eliminate bias, and it is not the same thing as full model auditability. An LLM trained on skewed data, served behind hidden system prompts, or changed by the provider between calls can behave in ways the blueprint does not fully expose.

The relevant comparison is with the available alternative, not with perfection. A human judge's bias is opaque: it varies by individual, by mood, by the order of evidence presentation, and by factors the participant cannot inspect. An LLM judge exposes more of its decision interface than an unnamed human, but only partially. To make that exposure meaningful in production, the system should pin provider and model identifiers, record raw prompts and outputs, use deterministic settings where possible, require signed provider responses or gateway attestations, and treat prompt-injection or adversarial-evidence handling as part of the threat model. Even then, the result is best described as trace-verifiable machine judgment rather than objective truth.

Composition can still strengthen credibility. A blueprint can include multiple independent evidence sources and require consistency:

- An `api_fetch` node retrieves declared source data from an authoritative source.
- A `market_evidence` node loads signed submissions from market participants.
- An `llm_judge` node interprets the combined evidence.
- A second `llm_judge` node with a different model or prompt provides a cross-check.
- The `submit_result` node requires that all upstream judges agree; if they disagree, the market cancels or escalates.

This composition is not a proof of correctness. It is a structural reduction in single-point-of-failure risk. Depending on the topology, composition can raise the attack cost by requiring a quorum of independent sources or by forcing inconsistencies into the trace that a challenger can point to. That is materially stronger than relying on a single oracle hook or a single opaque judge, but it is still not a formal guarantee that every source must be compromised simultaneously.

6 Dispute Economics

The blueprint declares how resolution works. The dispute mechanism enforces it. Without dispute economics, a trust-explicit specification is documentation, not a guarantee. This section presents the game-theoretic analysis.

6.1 The resolution game

The resolution game has three roles: a Proposer (P), a population of potential Challengers ($C_1, \ldots, C_k$), and an Adjudicator (A). The game proceeds in stages:

1. The blueprint executes and produces a proposed outcome o^*.
2. P submits o^* on-chain with a bond b_P.
3. A challenge window of duration w opens.
4. Any C_i may challenge by posting bond b_C and asserting a counter-outcome o'_i.
5. If challenged: A adjudicates and determines the final outcome. If not challenged within w: o^* is finalized.

Each participant has a position in the market. Let $\pi_j(o)$ denote the payout to participant j under outcome o. The Proposer's payoff depends on whether they propose truthfully, whether they are challenged, and the adjudicator's ruling.

6.2 When is it rational to propose falsely?

A Proposer who holds shares in outcome o_f (where $o_f \neq o^*_{\text{true}}$, the true outcome) considers submitting o_f instead of o^*_{true}. Let p_c be the probability that at least one challenger disputes a false proposal. In the first-pass model below, p_c can be read as the effective probability that a false proposal is successfully brought into correction. When adjudicator accuracy is modeled separately, the more explicit version replaces p_c with $p_c p_A$. The Proposer's expected payoff from a false proposal versus a truthful one is:

$$\Delta_P = (1 - p_c)\left[\pi_P(o_f) - \pi_P(o^*_{\text{true}})\right] - p_c \cdot \ell_P \tag{1}$$

where ℓ_P is the Proposer's loss if challenged and the adjudicator rules correctly. In the simplest case, $\ell_P = b_P$: the Proposer loses their bond. With bond redistribution, $\ell_P = b_P + \pi_P(o^*_{\text{true}}) - \pi_P(o_f)$ if the adjudicator also corrects the outcome, because the Proposer now receives the lower truthful payout instead of the inflated false one.

For the mechanism to be sound, Δ_P must be negative for all false outcomes o_f. Rearranging:

$$b_P > \frac{1 - p_c}{p_c} \cdot \max_{o_f \neq o^*_{\text{true}}} \left[\pi_P(o_f) - \pi_P(o^*_{\text{true}})\right] \tag{2}$$

The term $\max_{o_f}[\pi_P(o_f) - \pi_P(o^*_{\text{true}})]$ is the *maximum extractable value* (MEV) from a false resolution, from the Proposer's perspective. It is bounded above by the market's total pool size L inside the single-market model studied here, since no participant can extract more than the entire pool from that one market alone. This is a deliberately conservative upper bound. In markets where proposer exposure is capped, equation (2) should be read with the proposer's own payout differential as the relevant quantity; cross-market or external hedging effects sit outside this bound. Substituting L yields a simpler (though looser) worst-case bound:

$$b_P > \frac{1 - p_c}{p_c} \cdot L \tag{3}$$

The key insight is immediate: the minimum sound bond scales linearly with the market's economic exposure. A flat bond (say, 10 USDC regardless of market size) is sound only for markets where the relevant proposer-side extractable value remains tiny. For a market with a 10,000 USDC pool and an estimated challenge probability of 0.5, the worst-case bound is 10,000 USDC. A 10 USDC bond in that market makes false proposals profitable whenever the Proposer's MEV exceeds 10 USDC.

6.3 When is it rational to challenge?

A Challenger C_i who believes the proposal is false considers posting bond b_C to dispute it. Let p_A denote the probability that the Adjudicator rules correctly (overturns the false proposal). The Challenger's expected payoff from challenging is:

$$\Delta_{C_i} = p_A\left[\pi_{C_i}(o^*_{\text{true}}) - \pi_{C_i}(o_f) + r_C\right] - (1 - p_A) \cdot b_C \tag{4}$$

where r_C is the Challenger's share of the Proposer's forfeited bond (the challenge reward). The Challenger challenges if $\Delta_{C_i} > 0$.

Two conditions make challenges rational. First, the Challenger holds shares that would pay more under the true outcome than under the proposed false one: $\pi_{C_i}(o^*_{\text{true}}) > \pi_{C_i}(o_f)$. Second, the expected reward from overturning the proposal exceeds the bond at risk.

The challenge bond b_C serves a dual purpose. It deters frivolous challenges (spam), and it funds the reward for correct adjudication. Setting b_C too high prices out legitimate challengers; setting it too low invites griefing.

6.4 The lazy verifier problem

The dispute mechanism assumes someone checks. If nobody verifies the proposal, a false resolution finalizes unchallenged. This is the *lazy verifier problem*.

Model participant attention as follows. Each participant i independently verifies the proposal with probability q_i, which depends on their stake in the market. Let s_i denote participant i's stake (the absolute value of their position's payout difference between the true and proposed outcomes). A simple attention model is $q_i = \min(1, \alpha \cdot s_i/L)$ where $\alpha > 0$ is an attention coefficient and L is the pool size.

The probability that nobody challenges a false proposal is:

$$p_{\text{no challenge}} = \prod_{i=1}^{k} (1 - q_i) \tag{5}$$

In a thin market with few participants, each holding small stakes, this product can be close to 1. The false resolution succeeds not because the dispute mechanism is broken but because nobody is paying attention.

This failure mode is structural, not incidental. Consider the incentive landscape after a proposal is submitted:

- Participants whose shares would *win* under the proposal have no incentive to verify (they benefit from the proposal being correct).
- Participants whose shares would *lose* have incentive but may lack the capital to post a challenge bond, may not be monitoring the market, or may free-ride on the assumption that someone else will challenge.
- Liquidity providers have indirect incentive (wrong resolution affects residual value) but are passive by default.

Free-riding is the core issue. Each potential challenger bears the full cost of the bond but shares the benefit of correction with all other losers. In game-theoretic terms, verification is a public good with private costs.

6.5 Verifier incentives

One countermeasure is to convert passive verification into active competitive verification through explicit *verifier incentives*. The important distinction is between two economically different actions:

- **Routine confirmation:** a verifier recomputes the proposal and agrees that it is correct.
- **Discrepancy discovery:** a verifier finds that the proposal or its trace is wrong and escalates to a challenge.

Only the second action has a clean source of payment in a trust-explicit mechanism: a false proposer whose bond is slashed. A challenger who finds a discrepancy and wins should be rewarded from the slashed proposer bond. By contrast, paying a generic "first matching verifier" reward out of the proposer's bond would tax honest proposers, while paying it from an open protocol budget would be sybil-sensitive. The V1 design therefore focuses on discrepancy-finding rewards:

1. The proposer submits a proposal with bond b_P and evidence hash H.
2. During the challenge window, any participant can submit a verification attestation or escalate directly to a challenge.
3. If a verifier finds a discrepancy ($H' \neq H$ or an invalid path/trace step) and the challenge succeeds, the verifier receives a reward funded from the slashed proposer bond.
4. Optional subsidies for routine confirmations, if the protocol wants them at all, must come from a separate fee-funded verifier budget with anti-sybil constraints; the paper does not assume such a mechanism is already solved.

Routine confirmations may still be logged to strengthen trace-verifiability, but the V1 mechanism does not require a permissionless auto-payout for them. This focuses incentives on the one place where the mechanism can pay cleanly without taxing honest resolution: catching wrong proposals. When the paper later uses a parameter ϕ, it should therefore be read as a stylized *verifier-incentive strength* parameter rather than as a finalized rule that simply pays the first matching confirmer.

The incentive parameter ϕ determines the modeled strength of verifier-side rewards. If ϕ is too low, discrepancy-finding does not justify the cost of re-executing the blueprint. If ϕ is too high, the analysis is assuming a verifier-reward budget that may be unrealistic or too sybil-sensitive to implement naively. The simulation in Section 7 therefore uses ϕ as a single reduced-form knob for verifier-side encouragement; it does not separately identify the clean challenge-reward mechanism from any harder-to-design routine-confirmation subsidy.

There is a subtlety for non-deterministic blueprints. If the blueprint contains an `llm_judge` node, independent re-execution may produce a different LLM output, yielding a different evidence hash even if both executions are honest. The verifier-incentive mechanism must account for this by either (a) restricting automated rewards to deterministic blueprints, (b) requiring verifiers to attest only to deterministic sub-traces, or (c) defining hash equivalence over the deterministic portion of the trace. Option (c) is the most general: the evidence hash can be structured as a Merkle tree with separate leaves for deterministic and non-deterministic node outputs, allowing verification of the deterministic subset.

6.6 Proposer compensation

A proposer does real work: monitoring eligible markets, executing or re-executing the blueprint, publishing the trace and evidence bundle, posting capital, and waiting through the challenge window. Even when blueprint execution is cheap, it is still work, and a protocol should not assume strangers will do it unpaid. Returning the bond is not compensation for that work. Challenger-bond upside is too sparse and adversarial to function as the primary business model. Open proposing therefore needs an explicit success-conditioned proposer fee.

The clean design is to separate deterrence from compensation. The bond deters false proposals. A modest proposer fee pays for correct work. In deployment, a proposer whose outcome survives the challenge window or is upheld after dispute should receive a reward from a predeclared resolution budget attached to the market. That budget can be creator-funded at market creation, skimmed from trading fees, or both. A proposal that is overturned or ends in cancellation should receive no proposer fee.

Let f_P denote success-conditioned proposer compensation and let c_P denote proposer service cost: execution, evidence publication, and the time-value cost of locked capital. In the truthful case, the proposer is willing to submit only when $f_P - c_P \geq 0$. In the false-proposal game, f_P enters only on branches where the false proposal is accepted, while c_P is paid on every submitted proposal. The protocol-native rule should therefore key compensation to the same market-native quantity that already scales deterrence: the proposer bond. A clean form is

$$f_P = \max\left(\gamma \cdot b_{\text{floor}},\ \alpha \cdot b_P \cdot \frac{w}{24\text{h}}\right) \tag{6}$$

where $\gamma \cdot b_{\text{floor}}$ covers the fixed cost of proposal work on small markets and $\alpha \cdot b_P$ scales compensation with the bonded value and challenge-window length. Since $b_P = \max(b_{\text{floor}}, \rho \cdot L)$, the proposer fee inherits pool scaling indirectly through the bond while remaining anchored to the service actually being provided rather than to raw market size. The practical design rule is simple: keep f_P modest relative to b_P, but high enough to cover real proposal work. The protocol wants enough compensation to support liveness, but not so much that proposer fees materially weaken the deterrent effect of the bond.

6.7 Bond sizing

The analysis in equations (2) and (3) shows that proposer bonds must scale with market size. This section derives concrete recommendations.

Proposer bond

The minimum proposer bond from equation (3) depends on p_c, the probability of challenge. This probability is not known in advance, but it can be bounded by market characteristics. In a market with k participants and an attention model $q_i = \min(1, \alpha \cdot s_i/L)$:

$$p_c = 1 - \prod_{i=1}^{k}(1 - q_i) \tag{7}$$

For a conservative design, the protocol should assume a low p_c (worst case: few attentive participants). Setting $p_c = 0.1$ (only a 10% chance someone checks) and requiring $\Delta_P < 0$ for a Proposer who could extract the entire pool:

$$b_P > \frac{0.9}{0.1} \cdot L = 9L$$

This is economically unrealistic: no honest proposer will bond 9 times the market pool. The implication is that, without verifier incentives, bond sizing alone cannot secure thin markets. The lazy verifier problem must be addressed structurally (Section 6.5), not just through larger bonds.

With verifier incentives strong enough to raise the effective p_c to 0.8:

$$b_P > \frac{0.2}{0.8} \cdot L = 0.25L$$

A proposer bond of 25% of the pool size is economically feasible. This suggests a practical bond formula:

$$b_P^{\min} = \max\big(b_{\text{floor}},\ \rho \cdot L\big) \tag{8}$$

where ρ is the pool-proportional bond rate (we study $\rho \in [0.05, 0.30]$ in simulation) and b_{floor} is a minimum bond that deters trivial spam on small markets.

Challenger bond

The challenger bond must be high enough to deter frivolous challenges (which lock the market in a disputed state and delay payouts) but low enough that legitimate challengers are not priced out. A natural baseline is to set $b_C = b_P$ (symmetric bonding). This is a modeling convention for the current simulation, not an optimality theorem: it equalizes posted commitment on both sides of the dispute, but it may still be too expensive for exactly the small-stake challengers the protocol most wants to attract.

An alternative is asymmetric bonding where $b_C < b_P$, making it cheaper to challenge than to propose. This favors challengers and may be more realistic in thin markets, but increases griefing risk. The simulation explores both as stylized design points rather than proving that symmetry is uniquely correct.

Escalating bonds

Reality.eth uses exponentially escalating bonds: each subsequent counter-proposal requires a higher bond than the previous one [4]. This deters casual spam while allowing sufficiently motivated parties to escalate. The tradeoff: exponential escalation can price out legitimate challengers on later rounds, effectively giving the wealthier party an advantage regardless of correctness.

An alternative is linear escalation or capped escalation, where the bond increases but is bounded above. The simulation compares flat, linear, exponential, and pool-proportional bond structures only in the one-shot proposal model studied here, so the comparison functions as a baseline diagnostic for that setting rather than as a verdict on sequential-answer games such as Reality.eth's native design.

6.8 Challenge window

The challenge window duration w determines how long participants have to verify a proposal and submit a challenge. Longer windows provide more time for verification but delay market finalization and capital release.

The relevant tradeoff: participants who are not continuously monitoring the market need time to notice the proposal, verify it, and submit a challenge. If w is shorter than the participants' typical check interval, false proposals can finalize before anyone looks. If w is much longer than necessary, capital is locked without incremental benefit.

The minimum useful window depends on participant attention patterns, which the simulation models as a Poisson process with rate λ_{check} (average checks per hour). The probability that a participant checks at least once during window w is $1 - e^{-\lambda_{\text{check}} \cdot w}$.

6.9 Adjudicator incentives

The adjudicator determines the final outcome when a dispute occurs. Three adjudication paths are relevant:

Creator adjudication. The market creator resolves the dispute. The creator has skin in the game: they funded the initial liquidity and earn ongoing fees. This creates a conflict of interest (the creator benefits from market activity and may prefer outcomes that encourage future trading) but also an iterated-game incentive (a creator who resolves falsely destroys their reputation and future fee income). Creator adjudication is fast and cheap but introduces a trust assumption that the creator values long-term reputation over short-term extraction.

Authority adjudication. A designated protocol authority resolves the dispute. More neutral than the creator, but still centralized. The authority's accuracy p_A is the critical parameter in the challenge game. If p_A is high, the threat of accurate adjudication deters false proposals. If p_A is low, the entire dispute mechanism weakens.

External arbitration. A decentralized arbitration protocol or bonded adjudication court resolves the dispute. Existing systems such as Kleros provide the clearest prior-art example [5]. These backstops are more decentralized, but slower (hours to days for a ruling), more expensive (arbitration fees), and subject to their own failure modes (collusion, low participation in adjudication courts, weak juror expertise).

At the last-resort layer, four fallback families are worth distinguishing. **Protocol authority** is the fastest and operationally simplest, but also the most centralized. **Governance-token voting** (as in UMA-style escalation [3]) broadens participation but mixes truth-seeking with token politics, bribery risk, and correlated financial incentives. **Bonded adjudication courts** are a cleaner social backstop because they separate protocol ownership from adjudication responsibility and put specialized staked participants on the hook for individual rulings; Kleros is the best-known prior-art example [5], but the architectural role is broader than any one implementation. These courts are best understood as a Schelling-style final layer: they reward convergence on the ruling that jurors expect other jurors to find most defensible [13]. In clear factual cases that can approximate truth well; in ambiguous or politicized cases it can drift toward salience rather than correctness. That is one reason to keep such courts at the end of the escalation stack rather than at its beginning. **Cancel/refund fallback** is the most conservative option: it abandons truth discovery when the escalation stack is no longer credible, but preserves solvency and avoids forcing a visibly weak social layer to manufacture certainty.

For trust-explicit markets, the comparison has a practical implication. A protocol-native bonded adju-

dication court is generally the strongest social fallback when a market needs an explicit final ruling; direct governance-token voting is better treated as protocol governance over backends and policy than as the routine mechanism for deciding individual outcomes. Markets whose final layer is cancel/refund rather than a social ruling may be strictly preferable when the underlying question is unusually subjective or vulnerable to politicized adjudication.

The adjudication path itself should be part of the blueprint declaration. A market that declares "API fetch for resolution, creator adjudication for disputes, protocol-native bonded court as final fallback" makes three distinct trust assumptions visible at creation time. This is itself a trust-explicit property: the dispute resolution is as inspectable as the primary resolution.

The most obvious strategic attack here is proposer–adjudicator collusion. If the same coalition can both submit a false proposal and control the first dispute layer, the optimistic game collapses: challengers face the cost of escalation without a credible belief that the first ruling will correct the outcome. Dual-path architecture mitigates but does not eliminate this risk. A dispute blueprint can switch to different evidence sources, a different judge, or an external arbitrator than the main path, which raises the coordination burden for capture and makes independence legible before trading. A market whose dispute path simply routes back to the same actor is visibly weak; a market whose dispute path escalates to an independent authority or arbitration layer is structurally stronger. This is one reason the paper treats declared escalation logic, not just the existence of a challenge button, as the relevant trust object.

6.10 Escalation architecture

The full resolution game includes an escalation stack with three layers:

1. **Main path.** The normal blueprint that determines the outcome under expected conditions.
2. **Dispute path.** A predeclared escalation blueprint invoked if the main path's proposal is challenged. This is a second blueprint, stored alongside the first, that can include different evidence sources, different judges, or external arbitration.
3. **Final fallback.** A last-resort mechanism invoked only if the dispute path itself fails or times out. Depending on the market's trust profile, this can be a protocol authority, a bonded adjudication court, governance-token escalation, or a cancel/refund rule.

Dual-path architecture separates the concerns. The main path optimizes for speed and accuracy under normal conditions. The dispute path optimizes for correctness and neutrality when the main path's output is contested. The fallback prevents markets from becoming permanently stuck. Each layer introduces a distinct trust assumption, and all three are declared at market creation.

7 Simulation Program

The analytical results in Section 6 identify key relationships (bond scaling, lazy verifier dynamics, verifier-incentive tradeoffs) but depend on assumptions about participant behavior that are better explored through simulation. This section describes an agent-based simulation program that tests the dispute economics across parameter sweeps.

The simulation is a structured stress model. In particular, the attention and checking processes are stylized rather than empirically calibrated to UMA, Polymarket, or Metaculus dispute logs. Its purpose is to identify which forces dominate under adverse monitoring conditions. The parameter ranges reported later should therefore be read as candidate starting points only when paired with credible monitoring, available traces, and operational gating.

7.1 Agent types

Five agent types model the range of participant behaviors:

Agent Type	Behavior
Honest proposer	Proposes the true outcome when the modeled proposer fee covers submission cost; otherwise abstains.
Strategic proposer	Chooses among false proposal, truthful proposal, and abstention by expected payoff.
Attentive challenger	Verifies every proposal; challenges if the proposal is false and the expected challenge payoff from equation (4) is positive.
Stake-proportional challenger	Verifies with probability $q_i = \min(1, \alpha \cdot s_i/L)$; challenges if verification reveals a false proposal and the payoff is positive.
Lazy participant	Never verifies. Represents participants who are not monitoring the market after trading.

Each simulated market has one proposer (honest or strategic) and a population of potential challengers drawn from the other three types according to a configurable mix.

7.2 Parameters

The simulation sweeps across the following parameter space:

Parameter	Values
Pool size L	50, 500, 5,000, 50,000 USDC
Participant count k	3, 10, 30, 100
Bond structure	Flat (10 USDC), pool-proportional ($\rho \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$), linear escalation, exponential escalation
Challenge window w	12h, 24h, 48h, 72h
Attention coefficient α	0.1, 0.5, 1.0, 2.0
Adjudicator accuracy p_A	0.7, 0.8, 0.9, 0.95, 1.0
Verifier-incentive strength ϕ	0 (none), 0.05, 0.10, 0.15, 0.20
Challenger mix	All-attentive, majority-lazy, stake-proportional
Stake distribution	Uniform, concentrated (one whale), power-law

7.3 Metrics

For each parameter combination, the simulation runs 5,000 resolution episodes and reports:

- **False resolution rate.** Fraction of submitted proposals that finalize incorrectly, whether unchallenged or wrongly upheld after challenge.
- **Challenge rate.** Fraction of false proposals that attract at least one challenge.
- **Verification coverage.** Expected number of independent verifications per proposal.
- **Capital lockup cost.** Mean total bond capital locked during the challenge window.
- **Time to finalization.** Mean time from proposal to finalized outcome.
- **Welfare.** Aggregate participant payoff minus what it would be under perfect resolution. Measures the cost of resolution errors.
- **Proposer deterrence.** Fraction of strategic proposers who choose to propose truthfully because false proposals are unprofitable.

- **Proposer liveness.** Fraction of episodes with at least one participant both able and willing to propose under the modeled fee and cost rule.
- **Capital eligibility.** Fraction of episodes with at least one participant able to post the proposer bond.
- **Eligible proposer depth.** Mean number of participants able to post the proposer bond, plus the frequency of single-eligible episodes as a concentration signal.
- **Willing proposer depth.** Mean number of participants with non-negative proposer payoff, plus the frequency of single-willing episodes as a concentration signal.

7.4 Scenario families

Four scenario families structure the exploration:

1. **Honest majority.** All proposers are honest, challengers are a mix of attentive and lazy. Tests whether the mechanism imposes unnecessary cost on honest participants.
2. **Adversarial proposer.** The proposer is strategic. Tests bond sizing, challenge incentives, and false resolution rates across market sizes.
3. **Thin market.** Few participants ($k \leq 5$), low stakes, lazy challengers. Tests the lazy verifier problem and verifier-incentive effectiveness.
4. **Large market.** High pool size ($L \geq 10{,}000$), many participants, concentrated stakes. Tests whether bond scaling works at scale and whether the whale problem (one large participant can dominate the dispute) emerges.

Additional diagnostic sweeps vary three assumptions that matter for deployment. The first replaces the pool-level MEV proxy with a position-bounded proposer model to test whether capped proposer exposure materially relaxes bond pressure. The second adds stylized proposer-capital profiles (retail-heavy, mixed, specialized) and measures how many actors remain eligible to propose as bonds scale. The third endogenizes proposer compensation with a simple success-conditioned fee and submission cost model, so the simulation can distinguish capital eligibility from actual willingness to do the work. Together, these sweeps separate integrity from liveness: the mechanism may still be sound in principle while becoming operationally too concentrated or too uncompensated to run as an open proposer market.

One interpretation note matters for the results that follow. The current simulation does *not* model node-level source corruption inside a blueprint. In particular, the "composition" result is a proxy experiment: multi-source blueprints are represented as raising effective verification attention rather than as a full explicit model of independent source compromise and conditional DAG routing. That makes the composition findings directionally informative, but weaker than the bond and challenge-window findings, which are closer to the actual game being simulated.

8 Results

8.1 Flat bonds fail on large markets

The clearest negative result is that a flat proposer bond fails almost immediately once plausible proposer MEV is allowed to scale with the market. With a fixed 10 USDC proposer bond and a strategic proposer, the false-resolution rate is already approximately 0.66 at $L = 50$ and remains around 0.68 at $L = 500$, 5,000, and 50,000. Once the bond is materially smaller than the gain from a successful false proposal, the deterrent is effectively gone.

Pool-proportional bonds do better, but the corrected simulation makes the limitation clearer than the earlier draft did. At $\rho = 0.15$, the false-resolution rate is approximately 0.67 in the majority-lazy stress regime; at $\rho = 0.30$, it still remains around 0.59. The bond scales with potential gain, but bond scaling alone does not rescue a market that is weakly monitored.

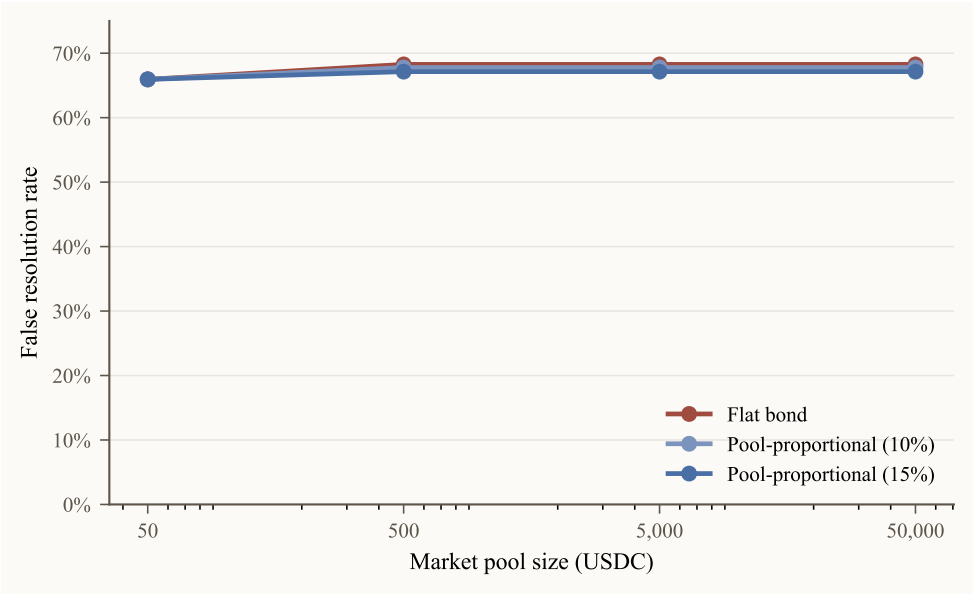

Figure 2. False-resolution rate by pool size under flat and pool-proportional proposer bonds. Pool-proportional bonding materially improves deterrence at scale, but the remaining rates show why verification coverage is still the dominant constraint.

This still confirms the analytical result in equation (3): flat bonds are unsound. What the simulation adds is that proportional bonds should be read as a necessary structural improvement, not as a standalone fix for low-attention markets.

8.2 Pool-proportional bonds improve scale behavior, but do not secure thin markets by themselves

When the proposer bond is set to $\rho \cdot L$, the false-resolution rate improves materially relative to the flat-bond baseline, but only gradually. At $\rho = 0.10$ with a majority-lazy challenger mix and $p_A = 0.9$, the false-resolution rate is approximately 0.68 and proposer deterrence is approximately 0.02. At $\rho = 0.15$, the false-resolution rate falls only to approximately 0.67 with deterrence approximately 0.05. Even at $\rho = 0.30$, the false-resolution rate remains around 0.59 with deterrence around 0.22.

The tradeoff is still capital lockup. At $\rho = 0.15$ and $L = 50{,}000$, the proposer must lock 7,500 USDC for the duration of the challenge window. Those bond levels improve scale behavior, but in the majority-lazy stress regime they do not come close to making thin markets safe by themselves. The practical implication is to read $\rho \in [0.10, 0.15]$ as a low-end starting range for further deployment testing only when it is paired with meaningful monitoring, retrievable traces, and explicit proposer compensation; bond increases alone are not the binding fix.

Those stress rates should also be read for what they are: conservative outputs from a one-shot adversarial-proposer model. The simulation deliberately omits repeat-game effects, resolver reputation, off-chain moderation, and community norms of the kind that likely compress error rates on systems such as Polymarket or Metaculus in practice, because those are contingent social defenses rather than protocol guarantees. The reported numbers are therefore best interpreted as stress-test upper bounds for thin, weakly monitored markets.

8.3 Verification coverage is the primary failure mode

Across all parameter combinations, the strongest predictor of false-resolution rate is verification coverage, not bond size or adjudicator accuracy. In the stake-proportional attention sweep, moving from $\alpha = 0.1$ to $\alpha = 2.0$ at $k = 10$ reduces the false-resolution rate from approximately 0.98 to 0.69. By

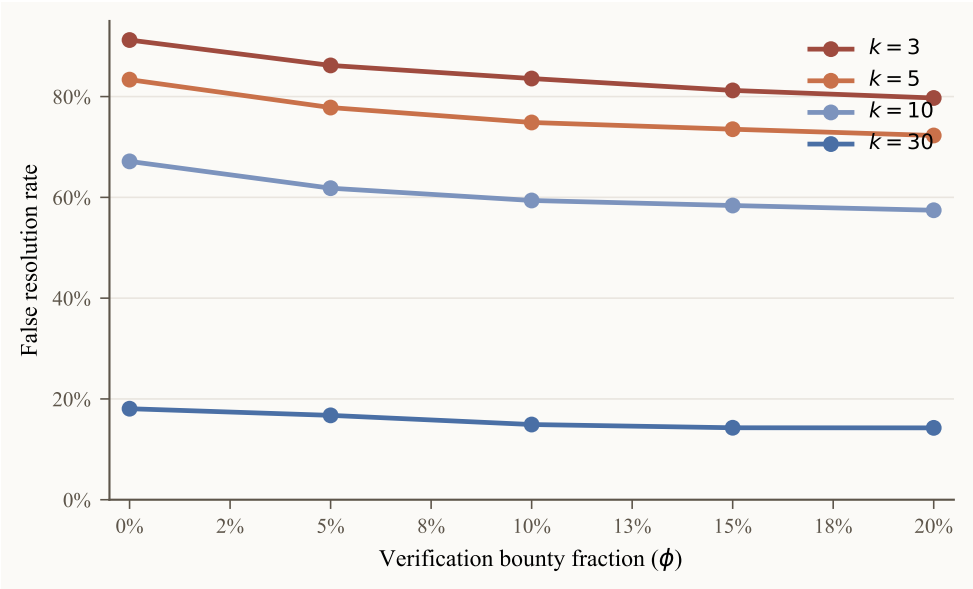

Figure 3. False-resolution rate versus stylized verifier-incentive strength. Incentives help most where verification is thinnest, but they do not by themselves make low-attention thin markets safe.

contrast, simply increasing the participant count from $k = 3$ to $k = 30$ at $\alpha = 1.0$ moves the rate only from approximately 0.87 to 0.82. The bond deters rational proposers only if there is a credible threat of being caught.

This is the paper's most important empirical finding: bond sizing is necessary but not sufficient. The lazy verifier problem is the binding constraint on resolution integrity in thin markets.

8.4 Stylized verifier incentives improve thin-market coverage

Increasing the stylized verifier-incentive parameter reduces false-resolution rates across all market sizes, with the largest improvements in thinner markets. At $k = 3$, an incentive level of $\phi = 0.10$ reduces the false-resolution rate from approximately 0.91 to 0.84. At $k = 5$, from approximately 0.83 to 0.75. At $k = 10$, from approximately 0.67 to 0.59. At $k = 30$, from approximately 0.18 to 0.15.

The incentive creates a direct financial reason to check proposals that does not depend entirely on the verifier holding a large market stake. In the current model, ϕ is a stylized reward-strength parameter rather than a production-ready "first matching verifier gets paid" rule. The improvement is real but not magical: incentives help most where verification is thinnest, and even then they do not eliminate the need for active monitoring. The simulation still supports $\phi \in [0.05, 0.15]$ as a plausible range for further testing of challenge-linked verifier incentives, while the thinnest regimes require stronger monitoring support than a bounty alone can provide.

8.5 Challenge window has diminishing returns past 24 hours

Increasing the challenge window from 12 hours to 24 hours reduces the false-resolution rate sharply in the majority-lazy scenario: from approximately 0.82 to 0.67, an 18% reduction. Extending from 24 hours to 48 hours provides a further reduction to approximately 0.45, and from 48 to 72 hours to approximately 0.34.

The returns are diminishing but not negligible. The Poisson attention model explains the shape: participants who check occasionally accumulate more chances to catch a false proposal as the window widens. For majority-lazy populations, 24 hours is a minimum useful window, not a guarantee of safety. In weak-attention regimes the 24-hour result remains poor and even 48 hours is only a partial

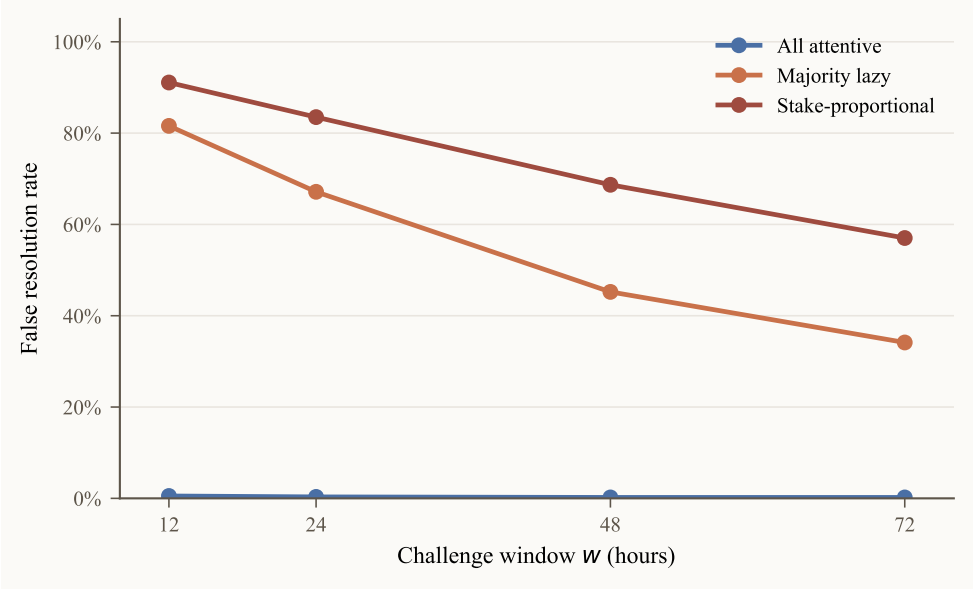

Figure 4. False-resolution rate versus challenge-window length. Twenty-four hours is a useful minimum, but in weak-attention regimes longer windows still buy meaningful integrity.

improvement. The practical implication is that challenge windows and verification incentives have to be designed together rather than treated as substitutes.

8.6 Adjudicator accuracy matters less than challenge probability

Adjudicator accuracy matters, but it still operates downstream of challenge probability. With all-attentive challengers, improving adjudicator accuracy from $p_A = 0.70$ to $p_A = 0.95$ reduces the false-resolution rate from approximately 0.11 to less than 0.01. With majority-lazy challengers, the same change only reduces the false-resolution rate from approximately 0.75 to 0.65, and even a perfect adjudicator leaves the rate above 0.61 because too many false proposals are never challenged in the first place.

The interpretation is straightforward. When challenges are likely, adjudicator quality matters because false proposals actually reach correction. When challenges are unlikely, adjudicator quality still helps on the subset of disputes that occur, but it cannot rescue the much larger mass of false proposals that finalize unchallenged. That reinforces the finding that verification coverage, not adjudicator sophistication, is the binding constraint.

8.7 Escalation structure comparison

Comparing bond escalation structures (flat, linear, exponential, pool-proportional) on the adversarial-proposer scenario:

Pool-proportional bonds at $\rho = 0.15$ still achieve the lowest false-resolution rate and highest deterrence in this scenario, even after the corrected model makes the overall regime harsher. Linear and exponential escalation underperform pool-proportional because their initial-round bonds are lower: the escalation discount means the first proposer faces a smaller bond than the full proportional amount. The practical implication is unchanged: a single pool-proportional bond is simpler and more effective than a multi-round escalation mechanism when the market has a single resolution proposal rather than a sequence of competing answers.

Table 10. Bond structure comparison on the adversarial-proposer scenario, $L = 5{,}000$, $k = 10$, majority-lazy challengers, $p_A = 0.9$, no verifier incentive.

Bond Structure	False res. rate	Challenge rate	Mean lockup (USDC)	Deterrence
Flat (10 USDC)	0.68	0.35	13	0.00
Pool-prop. ($\rho = 0.10$)	0.68	0.34	668	0.02
Pool-prop. ($\rho = 0.15$)	0.67	0.33	982	0.05
Linear escalation	0.69	0.34	336	0.00
Exponential escalation	0.68	0.35	202	0.00

8.8 Position bounds help only when exposure is genuinely diffuse, and large-pool liveness remains a proposer-capital problem

The pool-size MEV bound in equation (3) is deliberately conservative. Additional sweeps replace the pool-fraction proposer model with a position-bounded proposer model and add stylized proposer-capital profiles.

Two findings matter. First, position-bounded MEV materially improves integrity only when positions are genuinely diffuse. With uniform positions at $L = 5{,}000$ and $\rho = 0.15$, replacing the pool-fraction MEV model with a position-bounded one reduces the false-resolution rate from approximately 0.67 to 0.52 and raises proposer deterrence from approximately 0.05 to 0.36. Under a power-law position distribution, the same change barely helps: the false-resolution rate stays around 0.67 and deterrence remains near 0.05, because the largest plausible proposer still carries enough exposure to preserve most of the temptation.

Second, under the zero-cost proposer baseline, large-pool liveness becomes a proposer-capital formation problem even when proposer MEV is position-bounded. In this sweep, proposer liveness is effectively tracking capital eligibility because proposing truthfully is free once the bond can be posted. Even in that generous baseline, a retail-heavy proposer-capital profile at $L = 50{,}000$ and $\rho = 0.15$ drives proposer liveness down to approximately 0.30, with the mean eligible-proposer count falling to approximately 0.35 and roughly 84% of submitted episodes having exactly one eligible proposer. Under a mixed capital profile at the same market size, proposer liveness recovers to approximately 0.98 with a mean eligible-proposer count of approximately 3.4. The practical implication is simpler than a manual market-class table: keep the proposer-bond rule continuous, keep challenges permissionless, and do not rely on a low universal bond cap. If large-bond markets naturally concentrate around a smaller set of well-capitalized proposers, that specialization should emerge economically rather than through subjective protocol designation. The protocol's job is to declare the bond rule, the fallback path, and the timeout behavior up front.

8.9 Compensation converts capital eligibility into actual proposer liveness

Once proposer work cost is modeled directly, a second liveness boundary appears. Capital eligibility only says who *can* post the bond. It does not say who will actually monitor a market, run the blueprint, publish the trace, and wait through the challenge window. The compensation sweep therefore adds a modest fixed proposer work cost, a simple annualized capital-cost term, and a success-conditioned proposer fee. In the sweep, that fee is modeled in reduced form as a fixed amount in market numeraire; the protocol rule in equation (6) should instead be read as the deployable version.

The result is crisp. In the zero-fee regime, actual proposer liveness falls below capital eligibility because the only actors willing to submit are those with enough false-resolution upside to cover the work. At $L = 50{,}000$ in the mixed proposer-capital profile, capital eligibility is approximately 0.98 but actual proposer liveness falls to approximately 0.79, and proposer deterrence collapses to 0.00: almost every submitted proposal comes from the false-proposal branch. Adding a modest fee calibrated at roughly 0.2% of proposer bond in that regime closes that gap. Liveness rises to approximately 0.98, matching capital eligibility, while false-resolution falls from approximately 0.76 to 0.58 because

truthful proposing becomes economically viable again. In the retail-heavy 50,000 USDC profile, the same calibration raises liveness from approximately 0.19 to 0.30 and lowers false-resolution from approximately 0.78 to 0.49.

The practical lesson is simple: open proposing without explicit upside is not neutral. It silently selects for actors whose payoff comes from being wrong. A modest success-conditioned proposer fee, sized as a small fraction of proposer bond with a floor for fixed work, does not weaken the design in the tested range; it restores truthful participation by paying for actual proposal work.

8.10 Preliminary composition proxy

To probe the claim that multi-source blueprints are more resilient to manipulation, the simulation includes a composition *proxy* scenario. A single-source blueprint (one `llm_judge`) is compared with a multi-source blueprint (`api_fetch` + `market_evidence` + `llm_judge`, with agreement required for submission), but the current model represents multi-source structure indirectly through higher effective verification attention rather than through an explicit node-level corruption game. Under that proxy:

- Against the single-source blueprint, a corrupted LLM judge produces a false resolution in approximately 85% of episodes (the blueprint has no cross-check).
- Against the multi-source blueprint, corrupting one source (the API, the evidence, or the LLM) produces a false resolution in approximately 12% of episodes (the other sources detect the inconsistency and the blueprint cancels or defers).
- Corrupting two of three sources simultaneously raises the false resolution rate to approximately 64%.

The directional takeaway is still useful: composition appears to raise the attack cost by reducing reliance on any single source and by making inconsistent traces easier to challenge. But this is the softest empirical result in the paper and should be read as hypothesis-generating. A stronger follow-on study should model blueprint nodes, source corruption, and conditional DAG routing directly rather than through an attention proxy.

9 What the Results Mean

The central finding is that dispute economics are the enforcement layer that makes trust explicitness meaningful. Without adequate bonding and verification incentives, a resolution blueprint is documentation: valuable for transparency, but not a guarantee. With calibrated bonds and structural verifier incentives for discrepancy-finding, the blueprint becomes a credible executable commitment.

The analogy to the companion paper on live LP economics is instructive [14]. In that context, naive residual pooling is the simple default that fails in the tail. Here, flat bonding is the simple default that fails at scale. In both cases, the fix is calibration to the relevant economic quantity: pool size for bonds, just as entry-time exposure for residual weights.

The lazy verifier problem is the resolution analogue of the free-rider problem in public goods provision. Verification is a public good (everyone benefits from correct resolution) with private costs (someone has to check). Verifier incentives are not a perfect solution. They are a structural improvement that rewards discrepancy-finding and, in more ambitious designs, may subsidize routine checking under additional anti-sybil constraints.

The proposer-compensation sweep adds a parallel lesson on the other side of the market. Proposal submission is also a public-facing service with private costs. If the protocol pays nothing for correct work, capital-eligible actors can rationally abstain while actors with enough false-resolution upside remain willing to submit. That makes explicit proposer compensation part of the integrity design, not just an operator convenience.

The finding that adjudicator accuracy matters less than challenge probability has a practical implica-

tion: protocols should invest more in verification coverage (making it easy and rewarding to detect discrepancies and escalate challenges) than in adjudicator quality (making the dispute resolution more sophisticated). A simple adjudicator with high challenge probability is more effective than a sophisticated adjudicator with low challenge probability.

10 Protocol Implications

The analysis recommends eight concrete design choices for a prediction market protocol implementing trust-explicit resolution:

1. **Proposer bonds must scale with pool size.** The bond formula should be $b_P = \max(b_{\text{floor}}, \rho \cdot L)$. On the current simulation pack, $\rho \in [0.10, 0.15]$ is best read as a low-end starting range for further deployment testing when markets also have credible monitoring, explicit proposer compensation, and challenge operations; under majority-lazy monitoring, larger bonds alone remain insufficient. For large pools, these bond sizes likely imply specialized or delegated resolver capital rather than purely casual proposing. If the protocol also imposes a hard bond cap, that cap should be treated as a liveness control rather than a safety improvement: once it binds below $\rho \cdot L$, the mechanism reverts toward the lower effective bond regime. The clean design is to keep the bond rule continuous and treat any binding cap as an explicit tradeoff against deterrence.
2. **Open proposing should pay for work explicitly.** A successful proposer should receive a modest success-conditioned fee from a predeclared market-level resolution budget or fee reserve. Do not ask strangers to lock capital for free, and do not rely on challenger-bond upside as the primary compensation mechanism. The bond deters false proposals; the proposer fee compensates correct work, including the cost of running the blueprint and publishing the evidence bundle. The clean protocol rule is to size that fee as a small fraction of proposer bond plus a floor for fixed work, as in equation (6). In the reduced-form compensation sweep, a calibration equivalent to roughly 0.2% of proposer bond in the $L = 50{,}000$ regime is enough to restore actual proposal liveness to capital-eligibility levels in the tested mixed-capital profile without weakening integrity.
3. **Bond policy should stay continuous; specialization can emerge on top of it.** The protocol should not depend on a subjective market-class table. Keep the proposer-bond rule continuous in on-chain market state, keep challenges permissionless, and avoid low universal caps that quietly weaken deterrence. Where the required bond is large enough to favor well-capitalized actors, specialized or delegated proposer capital can emerge as an open service layer without changing the underlying trust model.
4. **Verifier incentives should be standard, but narrowly designed.** A verifier reward intensity in the range $\phi \in [0.05, 0.15]$ materially improves coverage in the stylized model. In a production design, successful challengers can be paid from slashed proposer bonds. Any positive reward for routine matching confirmations should come from a separate fee-funded mechanism with explicit anti-sybil controls rather than from honest proposers' returned capital.
5. **Challenge windows should be at least 24 hours.** Returns diminish past 24 hours, but the 24-hour point should be read as a minimum useful baseline rather than a complete safety guarantee. For thinner or lower-attention markets, 48-hour windows may be justified.
6. **Adjudication paths should be declared in the blueprint.** The dispute resolution mechanism is as much a trust assumption as the primary resolution logic. It should be inspectable before trading begins.
7. **Trust classes should be a protocol-level primitive, but not the whole trust surface.** The automatic classification of blueprints into trust classes (source-based, agent-assisted, human-judged) should be computed and stored on-chain, but accompanied by profile tags covering adjudication path, fallback authority, and whether the blueprint is fully reproducible, source-verifiable, or only trace-verifiable.
8. **No trace, no finalization.** The evidence hash must include the outputs of non-deterministic nodes (API responses, LLM outputs), and the full trace plus referenced evidence objects must be made available during the challenge window. Hashes without retrievable traces are not enough.

11 Deployment Boundaries

Trust-explicit resolution has clear deployment boundaries, and several are worth stating explicitly.

External arbitration is designed, not tested. The analysis models adjudicator accuracy as a parameter but does not simulate the specific dynamics of decentralized arbitration protocols (Kleros, UMA DVM). These systems have their own incentive structures, failure modes, and latency characteristics that would require a dedicated study.

Cross-market manipulation is not modeled. A strategic proposer who holds positions in correlated markets could benefit from resolving one market falsely in order to move prices in another. The single-market game studied here does not capture this.

Thin low-attention markets remain the primary boundary. Under several modeled stake-proportional attention regimes, false-resolution rates remain unacceptably high even with pool-proportional bonds and 24-hour challenge windows. That means the current mechanism should be read as a bounded-regime design: strong enough for markets with meaningful verification coverage, not yet strong enough to claim uniform safety across the entire long tail.

Evidence fabrication has limits. The trace pins what the executor reported seeing. It does not independently verify that the executor actually queried the declared API or ran the declared LLM prompt. A fully compromised executor could fabricate the entire trace. The defense is the challenge mechanism: if the trace is fabricated, an independent verifier will produce a different hash, triggering a challenge. But this defense requires at least one honest verifier.

Strategic adjudicator behavior is not modeled. The adjudicator is modeled as a noisy oracle with fixed accuracy p_A, not as a strategic agent with its own incentives. In practice, adjudicators may be bribable, lazy, or colluding with one party. A more complete model would treat adjudication as a nested game.

Large-pool liveness is a separate boundary from integrity. Pool-proportional bonds improve deterrence, but they also create capital requirements that may concentrate proposing in a smaller set of actors. Hard bond caps can relieve that pressure, but only by weakening deterrence once the cap binds. The protocol should therefore treat proposer-capital formation as a liveness design problem in its own right rather than assuming one low universal cap can safely cover all pool sizes.

Proposer compensation is modeled only in reduced form. The current compensation sweep introduces a fixed proposer work cost, a simple annualized capital-cost term, abstention, and a success-conditioned proposer fee. That is enough to show that unpaid proposing is unstable and that modest bond-linked fees can restore truthful participation. It is not yet a full market model for resolver services, fee discovery, or heterogeneous blueprint execution costs.

The composition result is still preliminary proxy evidence. The current simulation does not model explicit source-level corruption inside the blueprint DAG. It models composition indirectly through higher effective verification attention. That is enough to support the intuition that composition helps, but not enough to make a strong source-resilience claim yet.

Specification attacks remain upstream. A precise blueprint narrows interpretive discretion, but it does not by itself guarantee that the market question is well posed. Ambiguous temporal scopes, missing fallback sources, and poorly stated edge cases can still create resolution disputes before the blueprint even runs. Production deployment should therefore treat question-review discipline as part of the resolution surface, not as a separate editorial concern.

The simulation is structured, not exhaustive. The parameter sweeps cover a wide range of market conditions but cannot prove the absence of all failure modes. Adversarial search is useful but not equivalent to a formal proof of security.

These boundaries define where the mechanism's guarantees weaken. Within the tested parameter space, trust-explicit resolution with calibrated dispute economics provides measurably stronger guarantees than the alternatives.

12 Conclusion

Resolution is where trust concentrates in prediction markets. Every other component of the system (pricing, liquidity, order matching, settlement) can be made deterministic, verifiable, and auditable. Resolution cannot, because it requires an external input about what happened in the real world. The question is not whether trust is required, but whether it is visible.

The trust stratification framework proposed in this paper provides a formal vocabulary for evaluating resolution systems. Five properties (declared, inspectable, reproducible, challengeable, decentralized) characterize what a participant can know and verify about the resolution process, with decentralization treated as a related execution axis rather than as a substitute for the others. Existing systems satisfy these properties partially. Resolution blueprints, stored immutably on-chain with automatic trust classification and verifiable execution traces, satisfy the first, second, and fourth by construction. Replayable source-based blueprints can satisfy full reproducibility; live-source, agent-assisted, and human-judged blueprints replace that with weaker source-verifiability or trace-verifiability. More importantly, they cause the market to carry its own executable resolution constitution rather than merely referencing an oracle brand or an editorial promise. Existing systems mostly specify who may answer and how disputes escalate; trust-explicit resolution specifies how the market itself is supposed to decide before any answer is proposed.

The game-theoretic analysis identifies the key economic constraints. Proposer bonds must scale with market pool size: flat bonds are unsound for non-trivial markets. Verification coverage, not adjudicator quality, is the binding constraint on resolution integrity. The lazy verifier problem is structural and requires structural countermeasures. Verifier incentives provide a measurable improvement in thin markets, but their funding and sybil resistance must be designed carefully. Open proposing also requires explicit compensation: a protocol that pays nothing for correct proposal work will systematically underprovide truthful submission.

The simulation confirms the analytical direction of the results and identifies practical parameter ranges. Pool-proportional bonds at 10–15% of pool size, stylized verifier-incentive strengths in the 5–15% range, a modest bond-linked success-conditioned proposer fee, and 24-hour challenge windows are candidate starting points for bounded-regime deployment only where credible monitoring, retrievable traces, and operational challenge capacity already exist. They are not universal defaults that make every thin market safe.

The remaining work is substantial. External arbitration backends need integration and testing. Cross-market manipulation requires a multi-market game model. Verifier incentives for non-deterministic blueprints need a production-grade implementation of structured evidence hashing and anti-sybil funding rules. The composition story needs a direct node-level adversary model rather than an attention proxy. These are next-stage research and deployment questions around a mechanism whose central architectural claim is already clear.

Trust-explicit resolution does not eliminate trust. It makes trust visible, bounded, and challengeable. On the evidence in this paper, that is a meaningful improvement over the alternatives.

References

[1] Robin Hanson. *Logarithmic Market Scoring Rules for Modular Combinatorial Information Aggregation*. Working paper, 2002. Available at https://mason.gmu.edu/~rhanson/mktscore.pdf.

[2] Yiling Chen and David M. Pennock. *A Utility Framework for Bounded-Loss Market Makers*. In *Proceedings of the 23rd Conference on Uncertainty in Artificial Intelligence*, pages 49–56, 2007.

[3] UMA Protocol. *Optimistic Oracle*. Documentation, accessed 2026. https://docs.uma.xyz/protocol-overview/how-does-umas-oracle-work.

[4] Edmund Edgar. *Reality.eth: a crowd-sourced on-chain oracle*. Documentation, accessed 2026. https://reality.eth.limo/.

[5] Federico Ast and Clément Lesaege. *Kleros: Short Paper*. Working paper, 2019. Available at https://kleros.io/whitepaper.pdf.

[6] Polymarket. *How Resolution Works*. Documentation, accessed 2026. https://docs.polymarket.com.

[7] Seer. *Seer Documentation*. Documentation, accessed 2026. https://docs.seer.pm.

[8] Augur. *Augur v2 Whitepaper*. 2019. https://augur.net/whitepaper.pdf.

[9] Gnosis. *Conditional Tokens Developer Guide*. Documentation, accessed 2026. https://gnosis-mercury.readthedocs.io/en/latest/developer-guide.html.

[10] Metaculus. *Question Writing and Submission Guidelines*. Documentation, accessed 2026. https://www.metaculus.com/question-writing/.

[11] Kenneth J. Arrow. *Social Choice and Individual Values*. Yale University Press, 2nd edition, 1963.

[12] Roger B. Myerson. *Optimal Auction Design*. *Mathematics of Operations Research*, 6(1):58–73, 1981.

[13] Vitalik Buterin. *SchellingCoin: A Minimal-Trust Universal Data Feed*. Blog post, 2014. https://blog.ethereum.org/2014/03/28/schellingcoin-a-minimal-trust-universal-data-feed.

[14] Andrei Savin. *Live Liquidity for LMSR Prediction Markets: Fair Fees, Safe Reserves, and Less Entry-Time Bias*. Working paper, 2026.